

Assignment 3

Introduction to Computing For the Social Sciences

WS 2025/26 — **Deadline: 21.12.2025, 23:59**

Task 1: Improving an algorithm and compare runtimes

For this task imagine that you are working in a research group analyzing empirical conflict data ([ACLEDSouth-Sudan_2017.csv](#)). Your supervisor asked you to improve the efficiency of the code. After taking a look at the project's code you find the `sliding_window()` function:

```
def sliding_window(data, col_time, col_id, window_size):  
    """  
    add description  
  
    """  
    # initiate an empty list for the return  
    events_intime = []  
  
    for i in range(len(data)):  
        event_date = data[col_time].iloc[i]  
        events_intime.append([])  
  
        for j in range(len(data)):  
            other_date = data[col_time].iloc[j]  
  
            if abs((event_date - other_date).days) <= window_size and i != j:  
                events_intime[i].append(data[col_id].iloc[j])  
    return events_intime
```

1. Add the function description and add comments where you see it necessary.
2. Think about how you can make this function faster? Write a new function `sliding_window_optimized()`, which improves the runtime of the first function.
3. Systematically compare the runtime of the two variants (for different window-sizes) and visualize the results.

Note: Use the `event_date` column for timestamp and use the Python module `time` to track runtimes. The unoptimized version may run slowly on some machines, so aim to reduce how often you call it.

Task 2: Runtime of classic Sorting Algorithms

In this task, the runtime of two sorting-algorithms will be compared. Again, use the `time` package to measure the runtime.

a)

Implement the Selection Sort algorithm, as learned in the lecture, for a list input. Look up the algorithm behind Python's built-in sorting algorithm (`sorted()`). Describe it briefly in key points. (*Hint*: Python is using an algorithm called 'TimSort'). Compare the runtime of your own implementation of Selection Sort with the runtime of Python's built-in sorting algorithm on lists of different lengths (e.g., 1,000; 2,000; 3,000; up to 10,000) containing random integers, and visualize the results.

b)

Now test the runtime of (only) Python's `sorted()` function on lists ranging from 100,000 to 1,000,000 elements (in steps of 100,000). Test the following input cases:

- a list of random integers
- a list of integers sorted in descending order
- a list of integers sorted in ascending order

Again, please visualize the results.

Task 3: US Politician X-Posts - Preprocessing and Regular Expressions

In the assignment folder, you will find a `US_politican_tweets_pre_midterm_2022.zip` containing a `.json` file with tweets of US politicians. The file consists of 171,795 tweets that were tweeted by US politicians in the 5 months prior to the 2022 midterm elections (01.06.2022 - 08.11.2022, the election took place at 08.11.2022).

You will need to process this file aka. the tweets in it in order to work with the data in task 4 and the next assignment.

The posts have to be anonymized and cleaned. Create and apply these functions:

- `replace_mentions()`: a function that replaces all user mentions (`@username`) in the `text` column with `@user` (think of it as anonymizing the data). Use Regex and shortly explain how it works.
- `extract_hashtags()`: a function that extracts all hashtags (e.g. `#CDCWhistleblower`) appearing in the tweet and stores them in a new column `hashtags`. Use Regex and shortly explain how it works.

- `remove_urls()`: a function that extracts all domain names from the `expanded_url` field (`entities_urls` → `expanded_url`) to a new column (e.g. [google.com, wikipedia.org, ...]). Use Regex and shortly explain how it works.

Hint: Use a sample of tweets in order to test your preprocessing steps before you run them on the whole data set.

Task 4: STREAMLIT APP – MANUAL CODING

Using the **Streamlit** library, you are asked to develop a dashboard that supports the manual sentiment annotation of tweets. The basis is a random sample of **150 tweets** from the preprocessed data from task 3, saved as a `.json` file. Each entry must include the corresponding `tweet_id` so that your manual labels can later be compared to other machine generated labels, which we will introduce in assignment 4.

For labeling, always work with the **original text** of the tweets. Mentions should appear as `@user`, URLs should be removed, and hashtags should remain unchanged.

The dashboard should sequentially display random, unlabeled tweets and allow you to assign them to one of four categories: **positive**, **negative**, **neutral**, or **undecidable**. Once a tweet has been labeled, it is removed from the pool and the next unlabeled tweet is automatically shown. Previously labeled tweets must not be displayed again.

Another key functionality is the ability to save the current progress. The output file should not only contain tweets labeled during the current session but also those from earlier sessions.

The dashboard should be presented with a clear title, an intuitive interface and short notifications or confirmations after important actions (e.g., when saving). While aesthetics are not the main focus, a well-structured and user-friendly design is important.

On the technical side, `st.session_state` and callbacks will likely be useful. The *titanic_app* example from the tutorial may serve as inspiration. All code should be carefully commented so that the process and logic are transparent.